期末作业 • 前期调研

ThingsBoard 设备遥测与检修协同

行业现状、GitHub 类似方案与无 AI 系统流程图

关键词：ThingsBoard · 设备遥测 · 条件检修 · CMMS · 工单闭环

调研日期：2026-10-29

目录

核心结论 03

1. 行业现状 04

2. GitHub 类似系统与解决方案 07

3. 无 AI 系统流程图 12

4. 第一版 MVP 建议 15

参考链接 17

说明：目录页码为版式参考，最终页码以 Word 自动分页为准。

核心结论

1. 行业中的设备监测和检修管理通常由两类系统分别承担。ThingsBoard、Azure IoT、AWS IoT 等平台擅长设备接入、遥测存储、规则判断和可视化；Maximo、SAP EAM、Atlas CMMS 等系统擅长资产档案、工单、人员、备件和维护成本。
2. 真正影响检修效率的不是单一仪表盘，而是跨系统的状态同步和责任闭环。告警是否转成工单、工单是否有人负责、维修结果是否回写、告警是否经过恢复核验后关闭，是项目的核心价值。
3. GitHub 上可以找到 CMMS、预测性维护、ThingsBoard 遥测演示等分别成熟的项目，但“ThingsBoard 遥测 + 检修工单协同”的成熟高星一体化产品很少。最接近的仓库是一个低星、无许可证的集成服务，适合参考架构，不适合直接复制。
4. 无 AI 方案仍然具备完整业务价值。采用固定阈值、持续时间、去抖和人工确认，就可以实现条件检修闭环。AI 应作为后续增强项，而不是第一版系统的前置条件。
5. 期末项目的差异化应放在“跨系统协同”上，而不是重复实现 ThingsBoard 已有的设备接入、遥测曲线和基础告警能力。

1. 行业现状

1.1 检修模式正在从被动维修走向条件检修

工业设备维护通常经历由被动到主动的演进：

维护模式	主要做法	优点	局限
事后维修	设备故障后停机维修	实施简单、平时投入少	非计划停机多，维修成本不可控
定期预防维护	按日历或运行时长检修	计划性强，便于排班	可能过度维修，也可能漏掉突发劣化
状态检修	根据温度、振动、电流等阈值触发检修	比定期维护更贴近设备状态	依赖测点质量、阈值和告警治理
预测性维护	基于模型预测故障或剩余寿命	可提前安排资源	依赖历史数据、标注和模型治理

对本项目而言，无 AI 版本应明确落在“条件检修”而不是“预测性维护”。它可以依据固定阈值、持续超限时间和人工复核触发工单，不需要训练模型。这个边界既符合工程现实，也能避免把简单阈值规则包装成 AI 预测。

1.2 行业系统分工明确，但系统之间经常断开

当前常见系统可以分为三层：

层级	代表系统	主要职责
设备与遥测层	ThingsBoard、Azure IoT、AWS IoT、SCADA/ historians	设备接入、协议适配、遥测存储、规则处理、实时看板
资产与检修层	IBM Maximo、SAP EAM、Atlas CMMS、BaseEAM	资产台账、检修计划、工单、人员、备件、成本、审计
协同集成层	自研服务、iPaaS、消息总线、Webhook/API	设备与资产映射、告警转工单、状态同步、幂等与追踪

大型企业往往采用成熟 EAM/CMMS 加工业物联网平台，中小企业则更多使用 SaaS CMMS、低代码平台或 Excel 过渡。无论规模大小，设备台账与检修台账的主数据不一致、接口不稳定、告警和工单无法互相追踪，都是常见问题。

1.3 当前行业的主要痛点

1. 设备身份不统一。ThingsBoard 使用设备 ID，检修系统使用资产编号，两者缺少稳定映射。
2. 告警噪声大。同一测点反复越限会产生大量重复告警，需要持续时间、去抖、聚合和抑制机制。
3. 告警与工单脱节。告警只显示在监控大屏，没有责任人、处理时限和维修记录。
4. 工单重复创建。多个告警或重复回调可能为同一故障创建多张工单。
5. 维修结果不能回写。平台不知道工单是否已经完成，检修系统也不知道设备是否恢复。
6. 闭环条件不清。工单完成不等于设备恢复，告警也不应在维修提交后立即关闭。
7. 跨系统追踪困难。缺少统一 correlation ID、事件版本和审计时间线。
8. OT/IT 安全边界复杂。现场协议、网关、云平台和业务系统需要不同的凭据与网络策略。

1.4 近期行业趋势

1. 从单个平台走向组合式架构。物联网平台负责设备数据，CMMS/EAM 负责维护业务，中间通过 API 和事件协同。
2. 从“看数据”走向“处理事件”。规则告警需要进一步转化为工单、通知、升级和验收。
3. 条件检修仍是大规模落地的基础。相比模型预测，固定阈值加人工判断更容易解释、验收和运维。
4. 移动检修和现场反馈受到重视。维修人员需要在手机或平板上接单、填写记录、上传结果。
5. 标准化和审计要求提高。ISO 55000 强调资产管理，ISO 14224 强调设备可靠性与维护数据标准化，系统需要保留清晰的工单和状态变更记录。
6. 开源方案在试点和中小企业中增长。开源降低了部署门槛，但许可证、成熟度和长期维护仍需评估。

1.5 对本项目的启示

- ThingsBoard 不应被替代。它负责设备接入、遥测、规则链、告警和看板，是本项目的数据基础。

- 检修协同服务应成为独立模块。它负责跨系统映射、事件处理、工单编排、幂等和状态同步。
- 第一版不依赖 AI。阈值、持续时间、去抖、人工确认和恢复核验足以形成完整演示。
- 评价指标应包含业务闭环。例如告警转工单率、重复工单数、平均响应时间、平均修复时间、一次修复率和恢复核验通过率。

2. GitHub 类似系统与解决方案

2.1 检索范围与判断标准

本次以 GitHub 仓库名称、描述和 README 为主，检索词包括 ThingsBoard、telemetry、maintenance、work order、CMMS、predictive maintenance、condition monitoring 等。统计时间为 2026-10-29，Star 数会随仓库变化。

判断一个项目是否“类似”，不只看是否出现 ThingsBoard，还要看它是否具备遥测、告警、工单、人员检修和状态闭环中的多个环节。

2.2 候选项目对比

项目	遥测接入	规则告警	工单管理	跨系统闭环	成熟度	与本题关系
ThingsBoard	有	有	无	可通过规则链/API 扩展	高	本项目的数据基础
platform-integration	有	部分	有	有	早期项目	最接近的集成服务
Atlas CMMS	无	无	有	需二次集成	高	成熟 CMMS 参考
ShuangAn 双安	有	有	有	有	中等	业务闭环和领域模型参考
Preventive-Maintenance-of-Agitator	有	有	无	无	演示原型	ThingsBoard 告警链路参考
BaseEAM	无	无	有	无	较老	EAM/CMMS 数据模型参考

2.3 最接近的集成服务：platform-integration

仓库：https://github.com/xiangzuodalao/platform-integration

该仓库描述为 CMMS、ThingsBoard 和预测性维护工作流的集成服务。根据 README 和代码结构，它包含租户与设备绑定、ThingsBoard 历史遥测读取、预测任务、内部风险状态、ThingsBoard Alarm 同步、CMMS 工单创建和工单状态轮询。

可借鉴点：

- 使用独立服务连接 ThingsBoard、预测服务和 CMMS，不直接共享数据库。
- 使用设备映射表维护 ThingsBoard 设备 ID 与业务设备 ID。
- 告警与工单通过 external_ref、correlation_id、event_version 等字段关联。
- 同一风险已有活动工单时不再重复创建。
- 工单完成后不立即关闭风险，仍需等待设备健康状态恢复。
- 使用 outbox、重试和状态轮询处理跨系统不一致。
- 技术栈为 Python 3.12、FastAPI、SQLAlchemy、PostgreSQL、Alembic。

限制：

- 约 0 Star，项目较新，缺少生产使用验证。
- 仓库没有许可证，不能把代码直接复制到可分发项目中。
- 业务假设较多，部分流程依赖它自己的 CMMS 和预测服务。

结论：适合作为架构、接口字段、状态机和验收思路的第一参考，但期末项目应自行实现并裁剪。

2.4 成熟 CMMS 参考 ：Atlas CMMS

仓库：https://github.com/Grashjs/cmms

Atlas CMMS 是目前较为活跃的开源自托管 CMMS，GitHub 约 851 Star。它支持工单创建、分派、优先级、工时、维修历史、设备台账、库存、采购、服务请求、权限、自动化和移动端。

可借鉴点：

- 工单状态、负责人、优先级、工时和历史的完整建模。
- 设备台账与检修记录的关联。
- 班长分派、维修人员接单和技术员工作台。

- 检修统计指标与报表。
- Docker 自托管和前后端分离的部署方式。

限制：

- 不是专门的 ThingsBoard 接入方案，需要额外开发集成。
- 使用 AGPL-3.0 许可证。若后续分发或网络提供服务，需要评估许可证义务。

结论：适合参考工单业务和界面信息架构，不建议把它改造成 IoT 平台本身。

2.5 预测性维护框架参考：ShuangAn 双安

仓库：https://github.com/jiangnanboy/shuangan

ShuangAn 是面向中国工业环境的 Python 预测性维护框架，采用 Apache-2.0 许可证。它包含 MQTT、OPC UA 数据接入，异常检测，趋势预测，设备、工单、告警、故障模式、备件和维护计划，并支持通过 CMMS、IoT、知识库和通信平台连接器组织业务流程。

可借鉴点：

- Asset、WorkOrder、Alarm、FailureMode、MaintenancePlan 的领域模型。
- 传感器数据到告警、工单、通知、升级的处理链。
- 工单生命周期和多种工单类型。
- 只读沙盒、写操作确认和连接器配置。
- 中文工业场景与国内通信平台适配。

限制：

- 项目包含 AI Agent 能力，与本次“无 AI 流程图”需要明确区分。
- 它不是 ThingsBoard 专用方案，接入层需要根据本项目重新实现。

结论：适合参考业务模型和扩展方向，但对第一版无 AI 系统应只采用其中的规则、工单和状态管理思想。

2.6 ThingsBoard 条件监测演示：Preventive-Maintenance-of-Agitator

仓库：https://github.com/VaishnaviDadmal/Preventive-Maintenance-of-Agitator

该项目演示了传感器到 Siemens PLC、OPC UA、Node-RED、MQTT、ThingsBoard 的数据链路，监测振动、轴承温度、电机温度、压力、转速、功率和累计能耗，并在 ThingsBoard 中形成实时趋势、仪表盘和告警。

可借鉴点：

- 现场协议到 MQTT 的网关模式。
- 多测点设备的数据组织方式。
- ThingsBoard 告警和仪表盘设计。
- 项目范围内明确标注“条件监测，不是完整预测性维护”。

限制：

- 没有检修工单、人员分派和恢复核验。
- 是仿真原型，阈值未经现场验证。

结论：适合参考数据采集和 ThingsBoard 部分，不适合作为完整检修协同系统。

2.7 传统 EAM/CMMS 参考：BaseEAM

仓库：https://github.com/baseeam/baseeam-web-app

BaseEAM 是较早的 .NET EAM/CMMS 项目，包含资产、维护、库存、人员、工作流、故障代码、审计和 KPI。项目已多年未更新，技术栈较老。

可借鉴点：

- 资产层级、工单工作流、人员与班组模型。
- 预防性维护、库存和故障代码。
- 审计日志和通知机制。

限制：

- 没有 ThingsBoard 集成。
- 技术栈陈旧，不适合直接作为当前项目基础。

结论：适合查阅传统 EAM/CMMS 的业务字段，不建议直接运行或改造。

2.8 调研结论

GitHub 上的项目呈“两头成熟、中间稀缺”的格局：

- 遥测平台和 CMMS 分别都有成熟项目。
- 预测性维护、条件监测演示很多。
- 能把 ThingsBoard 告警、资产映射、工单、人员检修和恢复核验串成闭环的项目很少。

因此，本项目应把主要工作量放在协同层：

1. 设备与资产映射。
2. 告警去重与事件接入。
3. 工单创建、分派、接单和维修记录。
4. 工单状态回写与审计时间线。
5. 基于后续遥测的恢复核验。
6. 告警和工单的最终关闭。

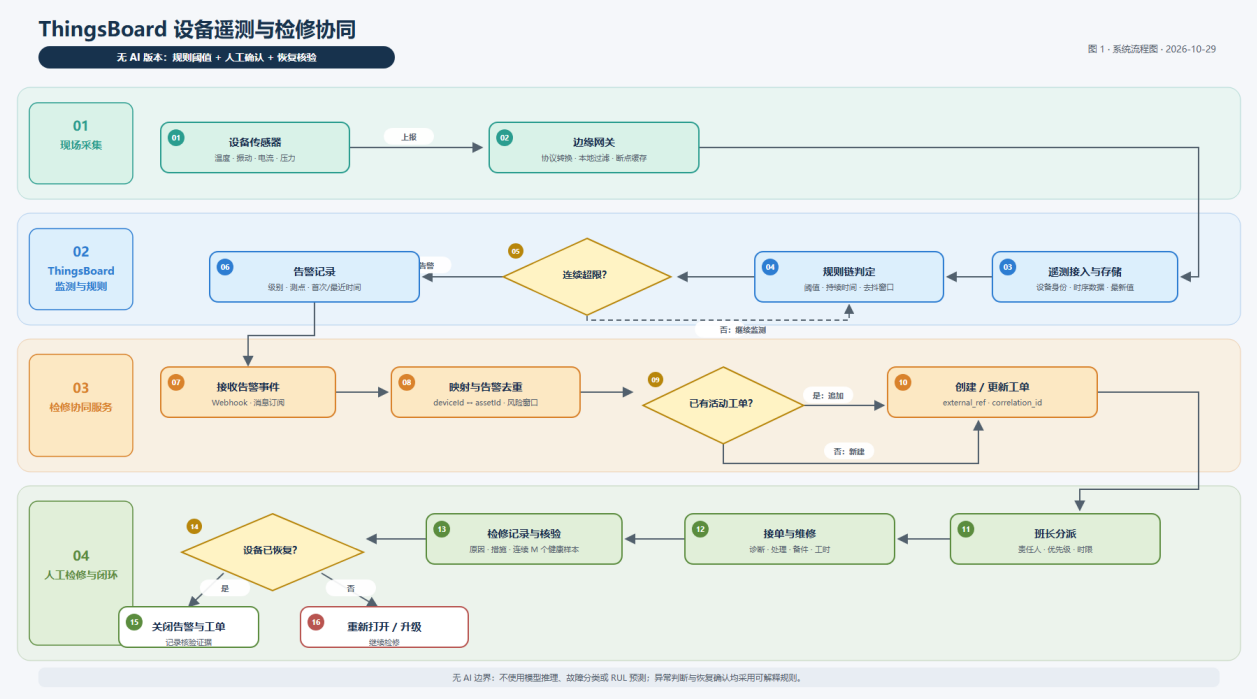
3. 无 AI 系统流程图

3.1 设计目标

无 AI 版本不使用模型推理、故障分类或剩余寿命预测，只依靠以下确定性规则：

- 固定阈值。
- 连续超限时长。
- 去抖与窗口聚合。
- 人工确认。
- 维修后连续健康样本核验。

系统的目标不是自动判断复杂故障，而是保证每一个有效异常都能被记录、分配、处理、验证和追溯。



无 AI 系统流程图

3.2 各层职责

层级	主要职责

层级	主要职责
现场设备与网关	采集温度、振动、电流等数据，通过 Modbus、OPC UA 等协议接入，并转换为 MQTT/HTTP 上报
ThingsBoard	设备管理、遥测存储、规则链判断、告警生成、看板和通知
检修协同服务	接收告警事件、设备与资产映射、告警去重、工单创建与去重、状态同步、审计
检修人员	班长分派、维修人员接单、现场维修、填写记录、提交结果
恢复核验	根据维修后的连续遥测判断是否真正恢复，决定关闭或重新打开

3.3 关键判断规则

1. 异常判断：测点超过阈值并持续 N 个采样周期，避免瞬时尖峰产生告警。
2. 告警去重：同一设备、同一测点、同一风险窗口内只保留一个活动告警。
3. 工单去重：同一告警或同一设备测点已有活动工单时，只更新工单上下文，不重复创建。
4. 人工确认：维修人员提交完成后，工单进入“待核验”，不能直接视为故障消除。
5. 恢复判断：维修后连续 M 个采样周期处于正常范围，才允许关闭告警。
6. 未恢复处理：核验不通过时重新打开工单或升级给班长。

3.4 工单状态机

建议采用以下最小状态机：

```
OPEN -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> COMPLETED -> VERIFIED -> CLOSED
```

异常分支：

- ON_HOLD：等待备件或外部条件。
- REOPENED：维修后核验未通过。
- CANCELLED：误报或重复工单，必须填写原因。

状态变化必须记录操作人、时间、原因和关联告警，形成不可丢失的审计时间线。

4. 第一版 MVP 建议

4.1 功能边界

第一版建议只做以下功能：

1. ThingsBoard 设备管理、遥测模拟和规则告警。
2. 设备与资产一对一映射。
3. 告警事件接入与去重。
4. 工单创建、分派、接单、维修记录和状态流转。
5. 维修后遥测恢复核验。
6. 告警关闭、工单关闭和审计时间线。
7. 设备状态、活动告警、待处理工单和维修历史页面。

第一版暂不做：

- AI 故障预测。
- 复杂备件采购流程。
- 多工厂、多组织计费。
- 复杂排班和 SLA 自动升级。
- 完整移动 App，可先用响应式 Web 页面替代。

4.2 可演示闭环

1. 模拟设备通过 MQTT 上报温度或振动。
2. ThingsBoard 规则链在连续超限后产生告警。
3. 协同服务接收告警并映射到资产。
4. 系统检查是否存在活动工单。
5. 若无活动工单，则创建工单并推送给班长。
6. 班长分派给维修人员，维修人员接单。

7. 维修人员填写原因、措施、备件和耗时并提交。
8. 系统继续接收遥测，执行连续健康样本核验。
9. 核验通过后关闭告警和工单；核验失败则重新打开并升级。
10. 页面展示完整时间线、响应时长、修复时长和恢复证据。

4.3 答辩时建议强调的创新点

- 不是再做一个 IoT 大屏，而是解决 ThingsBoard 与检修业务之间的断层。
- 工单完成不等于设备恢复，系统使用遥测进行二次验收。
- 通过设备映射、告警去重、工单去重和 correlation ID 解决跨系统一致性。
- 无 AI 版本具备完整可解释性，后续可以在不改业务闭环的前提下增加预测服务。

参考链接

1. ThingsBoard Rule Engine: https://thingsboard.io/docs/user-guide/rule-engine-2-0/overview/
2. ThingsBoard Alarms: https://thingsboard.io/docs/user-guide/alarms/
3. ThingsBoard GitHub: https://github.com/thingsboard/thingsboard
4. platform-integration: https://github.com/xiangzuodalao/platform-integration
5. Atlas CMMS: https://github.com/Grashjs/cmms
6. ShuangAn: https://github.com/jiangnanboy/shuangan
7. Preventive Maintenance of Agitator: https://github.com/VaishnaviDadmal/Preventive-Maintenance-of-Agitator
8. BaseEAM: https://github.com/baseeam/baseeam-web-app
9. IBM Maximo: https://www.ibm.com/products/maximo
10. SAP Asset Management: https://www.sap.com/products/asset-management.html